

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

To create a parent and child process using fork() and observe their PID, PPID, and process states using ps, top,

A and b

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    if (pid == 0) {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child sleeping...\n");
        sleep(10);
        printf("Child completed.\n");
        exit(0);
    }
    else {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        sleep(5);
        wait(NULL);
        printf("Parent completed.\n");
    }
    return 0;
}
Just b)
#include <stdio.h>
```

```
#include <unistd.h>
#include <stdlib.h>
int main() {
    printf("Process started. PID = %d\n", getpid());
    printf("Running...\n");
    for (long i = 0; i < 5000000000; i++) {
        // Busy work
    }
    printf("Going to sleep...\n");
    sleep(20);
    printf("Waking up...\n");
    return 0;
}
```

Output:

Parent Process

Parent PID : 98556

Child PID : 98557

Parent State: Waiting

Child Process

child pid:98557

parent PID : 98556

Child State: Running

Chil